Digital Nurture Design Patterns

Exercise 1: Singleton Pattern

Problem

Singleton ensures that only one object of a class exists throughout the application.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
class Logger{
 private static Logger instance;
 private Logger(){}
 public static Logger getInstance(){
  if(instance==null) instance=new Logger();
  return instance;
 }
 public void log(String msg){ System.out.println(msg); }
}
public class Test{
 public static void main(String[] args){
  Logger l1=Logger.getInstance();
  Logger l2=Logger.getInstance();
  System.out.println(l1==l2);
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 2: Factory Method Pattern

Problem

Factory Method creates objects without exposing the object creation logic to the client.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface Document{ void open(); }
class WordDocument implements Document{
 public void open(){ System.out.println("Word Document");}
}
class PdfDocument implements Document{
 public void open(){ System.out.println("PDF Document");}
}
```

```
abstract class DocumentFactory{
 abstract Document createDocument();
}
class WordFactory extends DocumentFactory{
 Document createDocument(){ return new WordDocument(); }
}
class Test{
 public static void main(String[] args){
  DocumentFactory f=new WordFactory();
  f.createDocument().open();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 3: Builder Pattern

Problem

Builder Pattern is used to construct complex objects step by step.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
class Computer{
 private String cpu,ram,storage;
 private Computer(Builder b){
  cpu=b.cpu; ram=b.ram; storage=b.storage;
 }
 static class Builder{
  String cpu,ram,storage;
  Builder setCPU(String c){cpu=c; return this;}
  Builder setRAM(String r){ram=r; return this;}
  Builder setStorage(String s){storage=s; return this;}
  Computer build(){ return new Computer(this);}
 }
 public String toString(){return cpu+" "+ram+" "+storage;}
}
class Test{
 public static void main(String[] args){
  Computer c=new Computer.Builder()
   .setCPU("i7").setRAM("16GB").setStorage("512GB SSD").build();
  System.out.println(c);
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 4: Adapter Pattern

Problem

Adapter Pattern allows incompatible interfaces to work together.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface PaymentProcessor{ void processPayment(int amount); }
class PayPalGateway{
 void makePayment(int amt){ System.out.println("PayPal: "+amt); }
}
class PayPalAdapter implements PaymentProcessor{
 private PayPalGateway gateway=new PayPalGateway();
 public void processPayment(int amount){ gateway.makePayment(amount); }
}
class Test{
 public static void main(String[] args){
  PaymentProcessor p=new PayPalAdapter();
  p.processPayment(500);
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 5: Decorator Pattern

Problem

Decorator Pattern adds new functionality to objects dynamically.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface Notifier{ void send(); }
class EmailNotifier implements Notifier{
 public void send(){ System.out.println("Email Sent"); }
}
abstract class NotifierDecorator implements Notifier{
 protected Notifier notifier;
 NotifierDecorator(Notifier n){ notifier=n; }
}
```

```
class SMSNotifierDecorator extends NotifierDecorator{
 SMSNotifierDecorator(Notifier n){ super(n); }
 public void send(){ notifier.send(); System.out.println("SMS Sent"); }
}
class Test{
 public static void main(String[] args){
  Notifier n=new SMSNotifierDecorator(new EmailNotifier());
  n.send();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 6: Proxy Pattern

Problem

Proxy Pattern controls access to another object and supports lazy loading.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface Image{ void display(); }
class RealImage implements Image{
 RealImage(){ System.out.println("Loading Image"); }
 public void display(){ System.out.println("Displaying Image"); }
}
class ProxyImage implements Image{
 RealImage img;
 public void display(){
  if(img==null) img=new RealImage();
  img.display();
 }
}
class Test{
 public static void main(String[] args){
  Image i=new ProxyImage();
  i.display(); i.display();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 7: Observer Pattern

Problem

Observer Pattern notifies multiple objects whenever the subject changes.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
import java.util.*;
interface Observer{ void update(float price); }
interface Stock{
 void register(Observer o);
 void notifyObservers();
}
class StockMarket implements Stock{
 List<Observer> list=new ArrayList<>();
 float price=100;
 public void register(Observer o){ list.add(o); }
 public void notifyObservers(){ for(Observer o:list) o.update(price); }
}
class MobileApp implements Observer{
 public void update(float p){ System.out.println("Price: "+p); }
}
class Test{
 public static void main(String[] args){
  StockMarket s=new StockMarket();
  s.register(new MobileApp());
  s.notifyObservers();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 8: Strategy Pattern

Problem

Strategy Pattern allows selecting an algorithm at runtime.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface PaymentStrategy{ void pay(int amt); }
class CreditCardPayment implements PaymentStrategy{
```

```
 public void pay(int amt){ System.out.println("Card: "+amt); }
}
class PaymentContext{
 PaymentStrategy p;
 PaymentContext(PaymentStrategy p){ this.p=p; }
 void execute(int a){ p.pay(a); }
}
class Test{
 public static void main(String[] args){
  new PaymentContext(new CreditCardPayment()).execute(1000);
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 9: Command Pattern

Problem

Command Pattern encapsulates requests as objects.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface Command{ void execute(); }
class Light{
 void on(){ System.out.println("Light ON"); }
}
class LightOnCommand implements Command{
 Light l;
 LightOnCommand(Light l){ this.l=l; }
 public void execute(){ l.on(); }
}
class RemoteControl{
 Command c;
 RemoteControl(Command c){ this.c=c; }
 void press(){ c.execute(); }
}
class Test{
 public static void main(String[] args){
  Light l=new Light();
  new RemoteControl(new LightOnCommand(l)).press();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 10: MVC Pattern

Problem

MVC separates application logic into Model, View and Controller.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
class Student{
 String name; int id; String grade;
 Student(String n,int i,String g){ name=n; id=i; grade=g; }
}
class StudentView{
 void display(Student s){
  System.out.println(s.name+" "+s.id+" "+s.grade);
 }
}
class StudentController{
 Student s;
 StudentController(Student s){ this.s=s; }
 void show(){ new StudentView().display(s); }
}
class Test{
 public static void main(String[] args){
  new StudentController(new Student("John",1,"A")).show();
 }
}
```

Output

Program executes successfully and demonstrates the respective design pattern.

Exercise 11: Dependency Injection

Problem

Dependency Injection supplies required dependencies from outside the class.

Setup

Create a new Java project with the name given in the exercise and add the classes below.

Java Implementation

```
interface CustomerRepository{
 void findCustomerById(int id);
```

```
}
class CustomerRepositoryImpl implements CustomerRepository{
 public void findCustomerById(int id){
  System.out.println("Customer "+id);
 }
}
class CustomerService{
 CustomerRepository repo;
 CustomerService(CustomerRepository r){ repo=r; }
 void get(int id){ repo.findCustomerById(id); }
}
class Test{
 public static void main(String[] args){
  CustomerService s=new CustomerService(new CustomerRepositoryImpl());
  s.get(101);
 }
}
```

Output

Program executes successfully and demonstr